Two Pointers

Sean

October 11, 2026



Outline

- When to think two pointers
- Opposite-direction pointers
- Sliding windows and sums
- Windows with frequencies
- Invariants and common mistakes



Introduction

- Two pointers reuses work while scanning pairs or contiguous ranges
- A safe move rules out candidates or restores a window invariant
- Count both pointer moves and the cost of updating the state



1 When to Think Two Pointers

2 Opposite-Direction Pointers

3 Sliding Windows and Sums

4 Windows with Frequencies

5 Invariants and Common Mistakes



When to Think Two Pointers



When to Think Two Pointers

- The input is sorted, or the task concerns a contiguous range
- A pointer can advance without revisiting discarded candidates
- Growing or shrinking the range changes a condition predictably
- Repeatedly checking every pair or subarray duplicates work



Counting Pointer Moves

- In an inward scan, each step shortens the remaining range

Sorting and preprocessing add their own costs.



Counting Pointer Moves

- In an inward scan, each step shortens the remaining range
- In a forward window, each pointer advances at most n times

Sorting and preprocessing add their own costs.



Counting Pointer Moves

- In an inward scan, each step shortens the remaining range
- In a forward window, each pointer advances at most n times
- A nested `while` loop can therefore do only $\mathcal{O}(n)$ total removals

Sorting and preprocessing add their own costs.



Counting Pointer Moves

- In an inward scan, each step shortens the remaining range
- In a forward window, each pointer advances at most n times
- A nested `while` loop can therefore do only $\mathcal{O}(n)$ total removals
- If each state update costs C, the scan costs $\mathcal{O}(nC)$

Sorting and preprocessing add their own costs.



1 When to Think Two Pointers

2 Opposite-Direction Pointers

3 Sliding Windows and Sums

4 Windows with Frequencies

5 Invariants and Common Mistakes



Opposite-Direction Pointers



Sorted Array Pair Search

- Start with `l = 0` and `r = n - 1` in a sorted array
- Compare `a[l] + a[r]` with the target
- If the sum is too small, discard the left endpoint
- If the sum is too large, discard the right endpoint

Use `l < r` so the two positions are different.



Why the Moves Are Correct

All candidate pairs still lie between l and r.

- If $a[l] + a[r] < x$, then for every $j \leq r$:

$$a[l] + a[j] \leq a[l] + a[r] < x$$

Each move discards an impossible endpoint.



Why the Moves Are Correct

All candidate pairs still lie between l and r.

- If $a[l] + a[r] < x$, then for every $j \leq r$:

$$a[l] + a[j] \leq a[l] + a[r] < x$$

- If $a[l] + a[r] > x$, then for every $i \geq l$:

$$a[i] + a[r] \geq a[l] + a[r] > x$$

Each move discards an impossible endpoint.



Two Sum in Code

```cpp
auto two_sum = [](const vector<int>& a, long long target) {
  int l = 0, r = (int)a.size() - 1;
  while (l < r) {
    long long sum = 1LL * a[l] + a[r];
    if (sum == target) return true;
    if (sum < target) ++l;
    else --r;
  }
  return false;
};
cout << two_sum({1, 2, 4, 7, 11}, 9);  // 1: true
cout << two_sum({1, 2, 4, 7, 11}, 10); // 0: false
// Require sorted input. The scan takes O(n) time.
```



Trace: Two Sum

Sorted values: $[1, 2, 4, 7, 11]$. Target: 9.

l	r	**Pair**	**Sum**	**Action**
0	4	$1 + 11$	$12 > 9$	Decrease r
0	3	$1 + 7$	$8 < 9$	Increase l
1	3	$2 + 7$	$9 = 9$	Found

We check three pairs rather than all $\binom{5}{2} = 10$ pairs.

Pair Search Boundaries

```cpp
// Uses two_sum from the previous code slide.
cout << two_sum({}, 8); // 0: empty
cout << two_sum({4}, 8); // 0: one position
cout << two_sum({4, 4}, 8); // 1: separate positions
cout << two_sum({-5, -1, 3, 8}, 2); // 1: -1 + 3
cout << two_sum({2000000000, 2000000000},
                4000000000LL);        // 1: wide sum
// Equal values may occupy different positions.
// Sorted order works with negative values too.
// 1LL widens the addition before int can overflow.
```

C++17 snippets assume `using namespace std;` and headers `<algorithm>`, `<vector>`, `<iostream>`, `<utility>`, and `<map>`.



Example: Sum of Two Values

CSES: Sum of Two Values

Find two different original positions whose values sum to x. $n \leq 2 \times 10^5$ and $x, a_i \leq 10^9$. Print the positions, or `IMPOSSIBLE` if no pair exists.

- Store each value together with its original position
- Sort the records, then run the pair scan
- Return the saved positions when a pair matches



Keeping Original Positions in Code

```cpp
vector<int> a = {2, 7, 5, 1}; // original order
int n = (int)a.size();
vector<pair<int, int>> v;    // {value, original position}
for (int i = 0; i < n; ++i) v.push_back({a[i], i + 1});
sort(v.begin(), v.end());
for (auto [value, position] : v)
  cout << value << ',' << position << ' ';
// 1,4 2,1 5,3 7,2
// Original positions are 1-based; sorted indices are not.
// Sorting the records leaves a unchanged.
```



Reporting a Pair in Code

```cpp
// Uses v and n from the saved-position slide.
long long target = 8;
int l = 0, r = n - 1;
pair<int, int> answer = {-1, -1};
while (l < r) {
  long long sum = 1LL * v[l].first + v[r].first;
  if (sum == target) {
    answer = {v[l].second, v[r].second}; break;
  }
  if (sum < target) ++l; else --r;
}
if (answer.first == -1) cout << "IMPOSSIBLE";
else cout << answer.first << ' ' << answer.second;
// 4 2; O(n log n) total time with sorting.
```



Counting Pairs with Sum $\leq k$

In a sorted array, suppose $a[l] + a[r] \leq k$.

- Every partner j with $l < j \leq r$ also works with l
- Add $r - l$ pairs, then advance l
- If the sum is too large, discard r as before

Count pairs of positions. Equal values still represent separate pairs.



Counting Pairs in Code

```cpp
vector<int> a = {1, 1, 2, 3, 4}; // already sorted
long long k = 4, count = 0;
int l = 0, r = (int)a.size() - 1;
while (l < r) {
  if (1LL * a[l] + a[r] <= k) {
    count += r - l;          // all partners l+1 ... r
    ++l;
  } else --r;
}
cout << count;               // 5 pairs of positions
// Scan: O(n). Sorting unsorted input adds O(n log n).
// At n=200000, n*(n-1)/2 can exceed a 32-bit int.
```



Trace: Counting Pairs

Values: $[1, 1, 2, 3, 4]$, $k = 4$.

l	r	**Sum**	**Count**	**Action**
0	4	5	0	Discard r
0	3	4	3	Add 3 pairs, advance l
1	3	4	5	Add 2 pairs, advance l
2	3	5	5	Discard r, then stop

Each pair has one smaller index, so advancing l counts it once.

1 When to Think Two Pointers

2 Opposite-Direction Pointers

3 Sliding Windows and Sums

4 Windows with Frequencies

5 Invariants and Common Mistakes



Sliding Windows and Sums



Sliding Windows

- Both pointers move forward through the original array
- Our variable window uses inclusive endpoints $[l, r]$
- Add the new right endpoint to the state
- Remove left endpoints until the window becomes valid

Keep the original order: sorting changes which values are contiguous.



Fixed and Variable Windows

Window	**Left endpoint**	**Purpose**
Fixed width w	$l = r - w + 1$	Length-w ranges
Variable width	Restore a rule	Valid ranges

A fixed-width sum window can include negative values.
An adaptive sum bound needs a monotonicity argument.

A Fixed-Width Sum Window in Code

```cpp
vector<int> a = {2, -1, 3, 4, -2};
int n = (int)a.size(), width = 3;
long long sum = 0;
for (int i = 0; i < width; ++i) sum += a[i];
long long best = sum;          // first complete window
for (int r = width; r < n; ++r) {
  sum += a[r];                 // value entering
  sum -= a[r - width];         // value leaving
  best = max(best, sum);
}
cout << best;                  // 6, from {-1, 3, 4}
// Sums of the three windows are 4, 6, 5.
// Require 1<=width<=n. O(n) time, O(1) extra space.
```



A Nonnegative Sum Invariant

Assume $a[i] \geq 0$ and $k \geq 0$. After shrinking:

- The current window has sum at most k
- Every earlier left endpoint gives a sum greater than k
- The current window is the longest valid one ending at r

Adding a nonnegative value cannot repair an earlier invalid window.
Removing one cannot increase the sum.



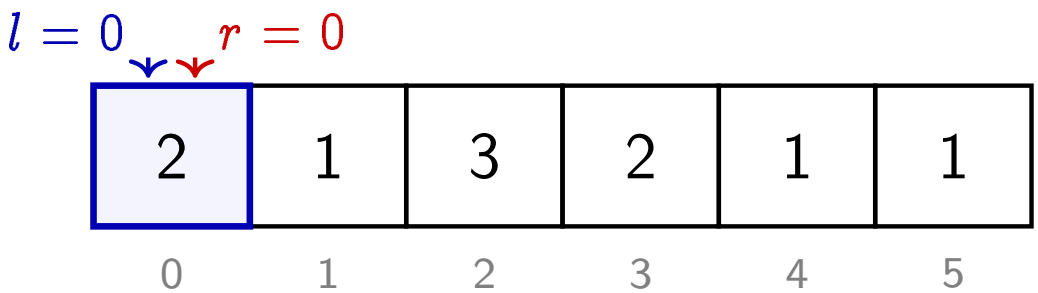
Longest Window with Sum $\leq k$

```cpp
vector<int> a = {2, 1, 3, 2, 1, 1};
long long k = 5, sum = 0;
int n = (int)a.size(), l = 0, best = 0;
for (int r = 0; r < n; ++r) {
  sum += a[r];
  while (l <= r && sum > k) {
    sum -= a[l]; ++l;
  }
  best = max(best, r - l + 1); // update after shrinking
}
cout << best;                  // 3: final window {2, 1, 1}
// Require a[i] >= 0 and k >= 0. Empty window: length 0.
// Each value enters and leaves at most once: O(n) time.
```



Sliding-Window Trace

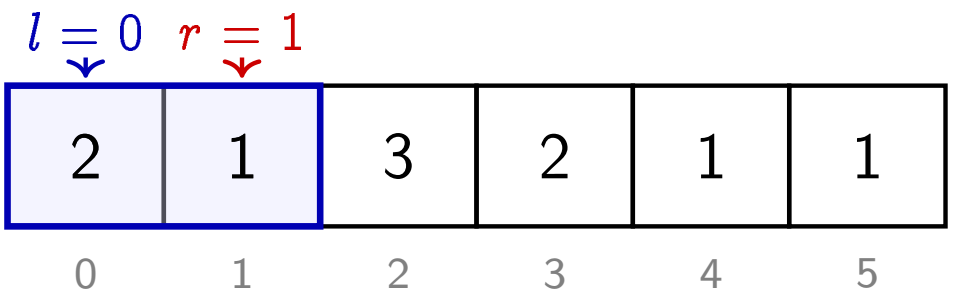
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Window $[0, 0]$, sum $= 2$, valid

Sliding-Window Trace

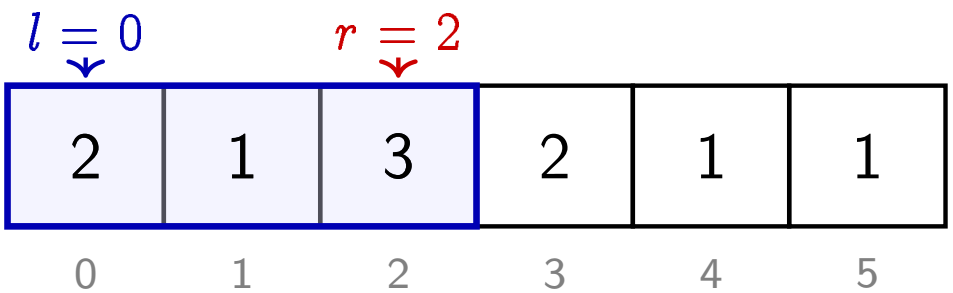
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Expand right: window $[0, 1]$, sum $= 3$, still valid

Sliding-Window Trace

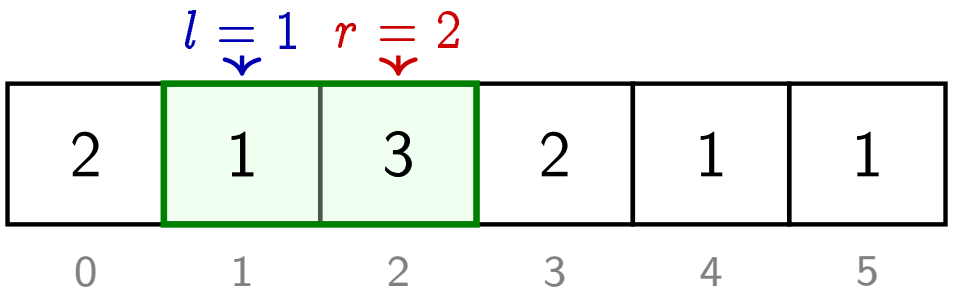
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Expand right: window $[0, 2]$, sum $= 6$, too large

Sliding-Window Trace

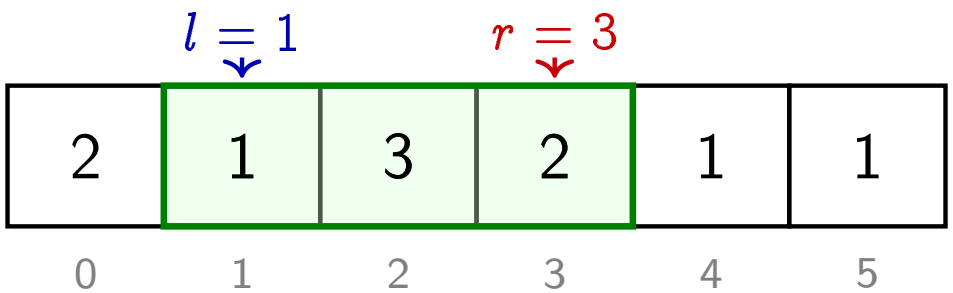
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Shrink left: window $[1, 2]$, sum $= 4$, valid again

Sliding-Window Trace

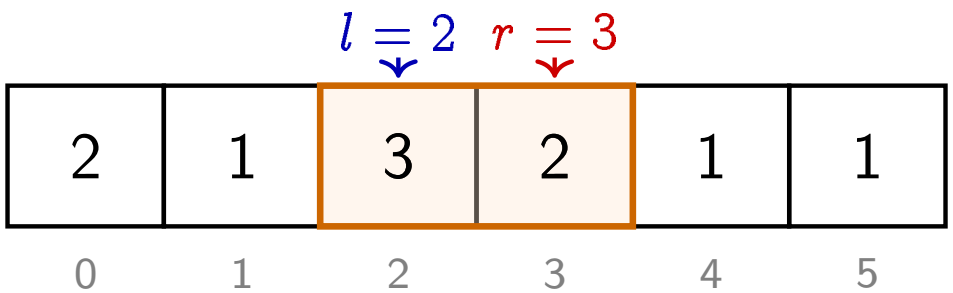
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Expand right: window $[1, 3]$, sum $= 6$, too large again

Sliding-Window Trace

Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Shrink left: window $[2, 3]$, sum $= 5$, valid

Trace: The Complete Window Scan

$a = [2, 1, 3, 2, 1, 1]$, $k = 5$. State after shrinking:

r	**New value**	l	**Sum**	**Length**	**Best**
0	2	0	2	1	1
1	1	0	3	2	2
2	3	1	4	2	2
3	2	2	5	2	2
4	1	3	3	2	2
5	1	3	4	3	**3**



Window Boundary Cases

For the longest nonnegative window with sum at most k:

Values	k	**Best length**
$[\,]$	5	0
$[0, 0, 0]$	0	3
$[8]$	5	0
$[5]$	5	1
$[1, 2]$	3	2

If $l = r + 1$, the window is empty and $r - l + 1 = 0$.



A Sum Window with Negative Values

$a = [5, -4, 3]$, $k = 4$.

r	**Greedy action**	**Remaining window**
0	$5 > 4$: remove the 5	Empty
1	Add -4	$[-4]$
2	Add 3	$[-4, 3]$, length 2

But the full array has sum 4 and length 3. The discarded 5 was needed. Use a technique based on prefix sums and the specific query condition.



Counting Valid Windows

For nonnegative sums at most k, shrink to the earliest valid l.

- Every suffix starting at $l, l+1, \ldots, r$ is also valid
- This gives $r - l + 1$ nonempty windows ending at r
- Add these counts for every right endpoint

Example: $[1, 2, 1]$, $k = 3$, contributes $1 + 2 + 2 = 5$ windows.



Counting Sum Windows in Code

```cpp
auto at_most = [](const vector<int>& a, long long k) {
  if (k < 0) return 0LL;
  int l = 0;
  long long sum = 0, count = 0;
  for (int r = 0; r < (int)a.size(); ++r) {
    sum += a[r];
    while (l <= r && sum > k) sum -= a[l++];
    count += r - l + 1;
  }
  return count;
};
cout << at_most({1, 2, 1}, 3); // 5
cout << at_most({0, 0}, 0);    // 3: zeros are allowed
// Require a[i] >= 0. Use long long for the count.
```



Example: Subarray Sums I

CSES: Subarray Sums I

Count subarrays with sum exactly x in an array of positive integers. $n \leq 2 \times 10^5$ and $1 \leq x, a_i \leq 10^9$.

- Add the next value, then shrink while the sum exceeds x
- If the sum equals x, count this window
- Strict positivity gives at most one matching start for each r



Counting Exact Positive Sums in Code

```cpp
vector<int> a = {2, 4, 1, 2, 7};
long long target = 7, sum = 0, count = 0;
int l = 0;
for (int r = 0; r < (int)a.size(); ++r) {
  sum += a[r];
  while (l <= r && sum > target) sum -= a[l++];
  if (sum == target) ++count;
}
cout << count;                    // 3: {2,4,1}, {4,1,2}, {7}
// Require a[i] > 0 and target > 0.
// O(n) time, O(1) extra space.
```



Exact Sums with Zeros

With zeros, several starts may give the same sum for one r. For $a = [0, 1]$ and target 1, index ranges $[0, 1]$ and $[1, 1]$ both work.

$$\text{exact}(x) = \text{atMost}(x) - \text{atMost}(x - 1)$$

```
// Uses at_most from the counting slide; values are >= 0.
cout << at_most({0, 1}, 1) - at_most({0, 1}, 0); // 2
cout << at_most({0, 0}, 0) - at_most({0, 0}, -1); // 3
// Integer sums: subtract counts below the target.
```

1 When to Think Two Pointers

2 Opposite-Direction Pointers

3 Sliding Windows and Sums

4 Windows with Frequencies

5 Invariants and Common Mistakes



Windows with Frequencies



Example: Playlist

CSES: Playlist

Find the longest contiguous sequence of different song IDs. $n \leq 2 \times 10^5$ and IDs are at most 10^9.

- Track the frequency of every ID in the current window
- After adding an ID, remove left values until its frequency is 1
- Update the best length after restoring distinctness

Keep the playlist order.



Playlist Frequencies in Code

```cpp
vector<int> a = {1, 2, 1, 3, 2, 7, 4, 2};
map<int, int> freq;
int l = 0, best = 0;
for (int r = 0; r < (int)a.size(); ++r) {
  ++freq[a[r]];
  while (freq[a[r]] > 1) {
    int old = a[l++];
    if (--freq[old] == 0) freq.erase(old);
  }
  best = max(best, r - l + 1);
}
cout << best;                    // 5: {1, 3, 2, 7, 4}
// map updates cost O(log n): O(n log n) total time.
// Pointer moves are O(n); extra memory is O(n).
```



Trace: Playlist

$a = [1, 2, 1, 3, 2, 7, 4, 2]$. At $r = 7$, remove $1, 3, 2$.

r	New ID	l	Length	Best
0	1	0	1	1
1	2	0	2	2
2	1	1	2	2
3	3	1	3	3
4	2	2	3	3
5	7	2	4	4
6	4	2	5	5
7	2	5	3	5



Last Occurrences in Code

```cpp
vector<int> a = {1, 2, 2, 1};
map<int, int> last;          // last processed index
int l = 0, best = 0;
for (int r = 0; r < (int)a.size(); ++r) {
  auto it = last.find(a[r]);
  if (it != last.end()) l = max(l, it->second + 1);
  last[a[r]] = r;            // update after the lookup
  best = max(best, r - l + 1);
}
cout << best;                // 2: {1,2} or {2,1}
// find() handles unseen IDs and index 0.
// max() keeps l moving forward; map: O(n log n).
```



An Old Occurrence Outside the Window

$a = [1, 2, 2, 1]$:

r	ID	Previous index	Old l	New l
0	1	Unseen	0	0
1	2	Unseen	0	0
2	2	1	0	2
3	1	0	2	$\max(2, 1) = 2$

At $r = 3$, setting l directly to 1 would include both copies of 2.



Example: Distinct Values Subarrays II

CSES: Distinct Values Subarrays II

Count nonempty subarrays with at most k distinct values.
$1 \leq k \leq n \leq 2 \times 10^5$.

- Keep frequencies of values in the current window
- Shrink until the number of distinct values is at most k
- Add $r - l + 1$ valid suffixes ending at r

Duplicate values are allowed here.



Counting Distinct-Value Windows in Code

```cpp
vector<int> a = {1, 2, 3, 1, 1};
int k = 2, l = 0;                 // require k >= 0
map<int, int> freq;               // only positive frequencies
long long count = 0;
for (int r = 0; r < (int)a.size(); ++r) {
  ++freq[a[r]];
  while ((int)freq.size() > k) {
    int old = a[l++];
    if (--freq[old] == 0) freq.erase(old);
  }
  count += r - l + 1;
}
cout << count;                    // 10; O(n log n) with map
// At k=0, every window shrinks to empty and adds 0.
```



Trace: At Most Two Distinct Values

$a = [1, 2, 3, 1, 1]$, $k = 2$:

r	l	**Distinct**	**Added**	**Count**
0	0	1	1	1
1	0	2	2	3
2	1	2	2	5
3	2	2	2	7
4	2	2	3	10

At $r = 4$, add three suffixes: $[1]$, $[1, 1]$, $[3, 1, 1]$.



Removing Zero Frequencies

```
map<int, int> freq = {{1, 1}, {2, 1}};
if (--freq[1] == 0) freq.erase(1);
cout << freq.size();        // 1: only ID 2 is in the
 ↪  window
// Decrementing alone would leave {1:0, 2:1}, size 2.
// erase() keeps size() equal to the number of live IDs.
```

A frequency map must describe the current window before we use `freq.size()` as the distinct count.



1 When to Think Two Pointers

2 Opposite-Direction Pointers

3 Sliding Windows and Sums

4 Windows with Frequencies

5 Invariants and Common Mistakes



Invariants and Common Mistakes



Common Mistakes

- Using pair pointers before sorting, or sorting a subarray task
- Reusing one position in a pair instead of requiring $l < r$
- Using one `if` when several left values need removing
- Updating the answer before the invariant is restored
- Moving l backward, or reading beyond an empty window



State Costs Matter

Each pointer moves $\mathcal{O}(n)$ times. The update cost determines the scan cost.

State	**Update**	**Total scan**
Running sum	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Frequency array of bounded IDs	$\mathcal{O}(1)$	$\mathcal{O}(n)$
`map` frequencies	$\mathcal{O}(\log n)$	$\mathcal{O}(n \log n)$

Large or sparse IDs need a map or preprocessing such as coordinate compression.



Choosing the Right Pattern

Task or input property	**Pattern**
A target pair in sorted values	Inward pair scan
Every window of width w	Add entering, remove leaving
Nonnegative sum bound	Grow and shrink a sum window
All values distinct	Frequencies or last occurrences
At most k distinct values	Frequencies and suffix counts
Static range sum queries	Prefix sums

Choose an invariant and justify every pointer move.



Takeaways

- Pointer movement needs a proof based on order or an invariant
- Preserve original positions for pair answers and original order for windows
- Longest windows use their length; counting windows adds valid suffixes
- Use wide types for sums and counts, and include state update costs



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

